TJ_modul_8

Nama : Helven Marcelyno Aldy Dwy Anggara

Nim : 108092500012

Kelas : IT-08-02

FILE CODE

1. queue_vaksinasi.h

queue_vaksinasi.h | queue_vaksinasi.cpp | main.cpp

```
#ifndef QUEUE_VAKSINASI_H
#define QUEUE_VAKSINASI_H

#include <iostream>
#include <string>

#define head(Q) ((Q).head)
#define tail(Q) ((Q).tail)
#define info(P) (P)->info
#define next(P) (P)->next
#define nill nullptr

using namespace std;

struct Infotype {
    string nama;
    int usia;
    string pekerjaan;
    bool prioritas;
    int nomor_antrean;
    int waktu_daftar;
    bool kondisi_darurat;
};

struct ElemQ {
    Infotype info;
    ElemQ *next;
};

struct Queue {
    ElemQ *head;
    ElemQ *tail;
};

void createQueue(Queue &Q);
bool isEmpty(Queue Q);
ElemQ* createElemQueue(string nama, int usia, string pekerjaan, int nomor_antrean, int waktu_daftar)
void enqueue(Queue &Q, ElemQ *P);
void dequeue(Queue &Q, ElemQ *&P);
ElemQ* front(Queue Q);
ElemQ* back(Queue Q);
int size(Queue Q);

void printInfo(Queue Q);
void serveQueue(Queue &Q);
void reassignQueue(Queue &Q);
void checkWaitingTime(Queue &Q, int waktu_sekarang);
void emergencyHandle(Queue &Q, int nomor_antrean);
void updatePriority(Queue &Q);
ElemQ* findAndRemove(Queue &Q, int nomor_antrean);

#endif

```

2. queue_vaksinasi.cpp

queue_vaksinasi.h | queue_vaksinasi.cpp | main.cpp

```cpp
#include "queue_vaksinasi.h"

void createQueue(Queue &Q) {
    head(Q) = nill;
    tail(Q) = nill;
}

bool isEmpty(Queue Q) {
    return head(Q) == nill;
}

ElemQ* createElemQueue(string nama, int usia, string pekerjaan, int nomor_antrean, int waktu_daftar) {
    ElemQ *P = new ElemQ;
    info(P).nama = nama;
    info(P).usia = usia;
    info(P).pekerjaan = pekerjaan;
    info(P).prioritas = (usia >= 60 || pekerjaan == "tenaga kesehatan");
    info(P).nomor_antrean = nomor_antrean;
    info(P).kondisi_darurat = false;
    info(P).waktu_daftar = waktu_daftar;
    next(P) = nill;
    return P;
}

void enqueue(Queue &Q, ElemQ *P) {
    if (isEmpty(Q)) {
        head(Q) = P;
        tail(Q) = P;
    } else if (info(P).prioritas) {
        if (!info(head(Q)).prioritas) {
            next(P) = head(Q);
            head(Q) = P;
        } else {
            ElemQ *temp = head(Q);
            while (next(temp) != nill && info(next(temp)).prioritas) {
                temp = next(temp);
            }
            next(P) = next(temp);
            next(temp) = P;
            if (next(P) == nill) {
                tail(Q) = P;
            }
        }
    } else {
        next(tail(Q)) = P;
        tail(Q) = P;
    }
}

void dequeue(Queue &Q, ElemQ *&P) {
    if (isEmpty(Q)) {
        P = nill;
        cout << "Semua warga telah terlayani." << endl;
    } else {
        P = head(Q);
        head(Q) = next(head(Q));
        if (head(Q) == nill) {
            tail(Q) = nill;
        }
        next(P) = nill;
    }
}

ElemQ* front(Queue Q) {
    return head(Q);
}

ElemQ* back(Queue Q) {
    return tail(Q);
}

int size(Queue Q) {
    int count = 0;
    ElemQ *temp = head(Q);
    while (temp != nill) {
        count++;
        temp = next(temp);
    }
    return count;
}

void printInfo(Queue Q) {
    if (isEmpty(Q)) {
        cout << "Antrean kosong." << endl;
    } else {
        cout << "Daftar Antrean:" << endl;
        ElemQ *temp = head(Q);
        while (temp != nill) {
            cout << "Nama: " << info(temp).nama << endl;
            cout << "Usia: " << info(temp).usia << endl;
            cout << "Pekerjaan: " << info(temp).pekerjaan << endl;
            cout << "Prioritas: " << (info(temp).prioritas ? "Ya" : "Tidak") << endl;
            cout << "Nomor Antrean: " << info(temp).nomor_antrean << endl;
            temp = next(temp);
        }
    }
}
```

```
}

void serveQueue(Queue &Q) {
    int count = 0;
    ElemQ *P;
    while (!isEmpty(Q) && count < 100) {
        dequeue(Q, P);
        if (P != nill) {
            cout << "Melayani warga:" << endl;
            cout << "Nama: " << info(P).nama << endl;
            cout << "Usia: " << info(P).usia << endl;
            cout << "Pekerjaan: " << info(P).pekerjaan << endl;
            cout << "Prioritas: " << (info(P).prioritas ? "Ya" : "Tidak") << endl;
            cout << "Vaksinasi berhasil." << endl;
            delete P;
            count++;
        }
    }
    if (count == 100 && !isEmpty(Q)) {
        cout << "Kapasitas telah penuh. Warga yang belum terlayani diminta kembali besok." << endl;
    }
}

void reassignQueue(Queue &Q) {
    Queue tempQ;
    createQueue(tempQ);
    ElemQ *P;
    while (!isEmpty(Q)) {
        dequeue(Q, P);
        next(P) = nill;
        info(P).prioritas = (info(P).usia >= 60 || info(P).pekerjaan == "tenaga kesehatan");
        enqueue(tempQ, P);
    }
    Q = tempQ;
}

void checkWaitingTime(Queue &Q, int waktu_sekarang) {
    Queue tempQ;
    createQueue(tempQ);
    ElemQ *P;
    while (!isEmpty(Q)) {
        dequeue(Q, P);
        next(P) = nill;
        if ((waktu_sekarang - info(P).waktu_daftar) >= 120) {
            info(P).prioritas = true;
        }
        enqueue(tempQ, P);
    }
    Q = tempQ;
}

void emergencyHandle(Queue &Q, int nomor_antrean) {
    Queue tempQ;
    createQueue(tempQ);
    ElemQ *P;
    ElemQ *darurat = nill;
    while (!isEmpty(Q)) {
        dequeue(Q, P);
        next(P) = nill;
        if (info(P).nomor_antrean == nomor_antrean && darurat == nill) {
            darurat = P;
            info(darurat).kondisi_darurat = true;
            info(darurat).prioritas = true;
        } else {
            enqueue(tempQ, P);
        }
    }
    Queue finalQ;
    createQueue(finalQ);
    if (darurat != nill) {
        enqueue(finalQ, darurat);
    } else {
        cout << "Warga dengan nomor antrean " << nomor_antrean << " tidak ditemukan." << endl;
    }
    while (!isEmpty(tempQ)) {
        dequeue(tempQ, P);
        next(P) = nill;
        enqueue(finalQ, P);
    }
    Q = finalQ;
}

void updatePriority(Queue &Q) {
    Queue tempQ;
    createQueue(tempQ);
    ElemQ *P;
    while (!isEmpty(Q)) {
        dequeue(Q, P);
        next(P) = nill;
        enqueue(tempQ, P);
    }
    Queue finalQ;
    createQueue(finalQ);
    Queue sisaQ;
    createQueue(sisaQ);
    while (!isEmpty(tempQ)) {
        dequeue(tempQ, P);
        next(P) = nill;
        if (info(P).kondisi_darurat) {
            enqueue(finalQ, P);
        } else {
            enqueue(sisaQ, P);
```

```
199             }
200         }
201         while (!isEmpty(sisaQ)) {
202             dequeue(sisaQ, P);
203             next(P) = nill;
204             enqueue(finalQ, P);
205         }
206         Q = finalQ;
207     }
208
209     ElemQ* findAndRemove(Queue &Q, int nomor_antrean) {
210         Queue tempQ;
211         createQueue(tempQ);
212         ElemQ *P;
213         ElemQ *found = nill;
214         while (!isEmpty(Q)) {
215             dequeue(Q, P);
216             next(P) = nill;
217             if (info(P).nomor_antrean == nomor_antrean && found == nill) {
218                 found = P;
219             } else {
220                 enqueue(tempQ, P);
221             }
222         }
223         Q = tempQ;
224         if (found == nill) {
225             cout << "Warga dengan nomor antrean " << nomor_antrean << " tidak ditemukan dalam antrean." << en
226         }
227         return found;
228     }
229
```

3. main.cpp

queue_vaksinasi.h | queue_vaksinasi.cpp | main.cpp

```cpp
#include "queue_vaksinasi.h"

int main() {
    Queue Q;
    createQueue(Q);

    ElemQ* P1 = createElemQueue("John Doe", 65, "lansia", 1, 0);
    ElemQ* P2 = createElemQueue("Alice", 30, "tenaga kesehatan", 2, 5);
    ElemQ* P3 = createElemQueue("Bob", 25, "pekerja", 3, 10);
    ElemQ* P4 = createElemQueue("Charlie", 70, "pensiunan", 4, 15);
    ElemQ* P5 = createElemQueue("David", 28, "pekerja", 5, 20);

    enqueue(Q, P1);
    enqueue(Q, P2);
    enqueue(Q, P3);
    enqueue(Q, P4);
    enqueue(Q, P5);

    cout << "Isi antrean awal:" << endl;
    printInfo(Q);

    cout << "\nMelakukan pelayanan pada antrean:" << endl;
    serveQueue(Q);

    cout << "\nisi antrean setelah pelayanan:" << endl;
    printInfo(Q);

    ElemQ* P6 = createElemQueue("Edward", 22, "pekerja", 6, 30);
    enqueue(Q, P6);

    cout << "\nMengatur ulang antrean berdasarkan prioritas:" << endl;
    reassignQueue(Q);
    printInfo(Q);

    cout << "\nMemeriksa waktu tunggu dan mengubah prioritas jika lebih dari 2 jam:" << endl;
    checkWaitingTime(Q, 150);
    printInfo(Q);

    cout << "\nMenangani kondisi darurat untuk warga dengan nomor antrean 5:" << endl;
    emergencyHandle(Q, 5);
    printInfo(Q);

    cout << "\nMengupdate prioritas antrean setiap jam:" << endl;
    updatePriority(Q);
    printInfo(Q);

    cout << "\nMenghapus warga dengan nomor antrean 3:" << endl;
    ElemQ* removedElem = findAndRemove(Q, 3);
    if (removedElem) {
        cout << "Warga yang dihapus: " << info(removedElem).nama << endl;
        delete removedElem;
    }
    printInfo(Q);

    cout << "\nUkuran antrean saat ini: " << size(Q) << endl;

    return 0;
}
```

BUILD & RUN

```
Isi antrean awal:
Daftar Antrean:
Nama: John Doe
Usia: 65
Pekerjaan: lansia
Prioritas: Ya
Nomor Antrean: 1
Nama: Alice
Usia: 30
Pekerjaan: tenaga kesehatan
Prioritas: Ya
Nomor Antrean: 2
Nama: Charlie
Usia: 70
Pekerjaan: pensiunan
Prioritas: Ya
Nomor Antrean: 4
Nama: Bob
Usia: 25
Pekerjaan: pekerja
Prioritas: Tidak
Nomor Antrean: 3
Nama: David
Usia: 28
Pekerjaan: pekerja
Prioritas: Tidak
Nomor Antrean: 5
```

```
Melakukan pelayanan pada antrean:
Melayani warga:
Nama: John Doe
Usia: 65
Pekerjaan: lansia
Prioritas: Ya
Vaksinasi berhasil.
Melayani warga:
Nama: Alice
Usia: 30
Pekerjaan: tenaga kesehatan
Prioritas: Ya
Vaksinasi berhasil.
Melayani warga:
Nama: Charlie
Usia: 70
Pekerjaan: pensiunan
Prioritas: Ya
Vaksinasi berhasil.
Melayani warga:
Nama: Bob
Usia: 25
Pekerjaan: pekerja
Prioritas: Tidak
Vaksinasi berhasil.
Melayani warga:
Nama: David
Usia: 28
Pekerjaan: pekerja
Prioritas: Tidak
Vaksinasi berhasil.
```

```
isi antrean setelah pelayanan:
Antrean kosong.

Mengatur ulang antrean berdasarkan prioritas:
Daftar Antrean:
Nama: Edward
Usia: 22
Pekerjaan: pekerja
Prioritas: Tidak
Nomor Antrean: 6

Memeriksa waktu tunggu dan mengubah prioritas jika lebih dari 2 jam:
Daftar Antrean:
Nama: Edward
Usia: 22
Pekerjaan: pekerja
Prioritas: Ya
Nomor Antrean: 6

Menangani kondisi darurat untuk warga dengan nomor antrean 5:
Warga dengan nomor antrean 5 tidak ditemukan.
Daftar Antrean:
Nama: Edward
Usia: 22
Pekerjaan: pekerja
Prioritas: Ya
Nomor Antrean: 6

Mengupdate prioritas antrean setiap jam:
Daftar Antrean:
Nama: Edward
Usia: 22
Pekerjaan: pekerja
Prioritas: Ya
Nomor Antrean: 6

Menghapus warga dengan nomor antrean 3:
Warga dengan nomor antrean 3 tidak ditemukan dalam antrean.
Daftar Antrean:
Nama: Edward
Usia: 22
Pekerjaan: pekerja
Prioritas: Ya
Nomor Antrean: 6

Ukuran antrean saat ini: 1
```